

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Problema/ Preocupación de diseño	El sistema requería una arquitectura que permitiera mantener independencia entre la capa de presentación y la lógica de negocio, facilitando el mantenimiento, escalabilidad y despliegue en distintos entornos (local o nube).
Orientación de la decisión	Adoptar una arquitectura cliente-servidor, donde el frontend se desarrolle en React y consuma los servicios del backend desarrollado en Django mediante API REST. Los elementos afectados son la interfaz de usuario, el servidor web y el flujo de comunicación entre cliente y servidor.
Supuestos	- Se cuenta con un entorno capaz de ejecutar Django y Node.js. - Los usuarios accederán al sistema desde navegadores modernos. - La comunicación entre frontend y backend se realiza mediante endpoints REST seguros.
Restricciones	- Comunicación obligatoria a través de HTTPS. - El frontend no puede acceder directamente a la base de datos. - Separación estricta entre capas lógicas (UI, lógica de negocio y persistencia).
Requerimientos relacionados	RF01: El sistema debe ofrecer una interfaz web dinámica y responsiva. RF02: El sistema debe permitir la interacción entre usuarios y el servidor mediante servicios web.
Alternativas consideradas	Alternativa 1: Sistema monolítico con Django (frontend y backend integrados). Alternativa 2: Arquitectura desacoplada con React y Django conectados por API REST.
Evaluación de alternativas	Alternativa 1 Ventajas: Simplicidad inicial. Desventajas: Dificultad para escalar, bajo aislamiento de responsabilidades. Alternativa 2 Ventajas: Escalabilidad, mantenibilidad, posibilidad de evolución a microservicios. Desventajas: Mayor complejidad de configuración inicial (CORS, despliegue).
Alternativa seleccionada y Justificación	Se elige la arquitectura desacoplada (React + Django) porque facilita la modularidad, el mantenimiento y la integración con servicios externos (como Azure SQL y autenticación en la nube).
Decisiones relacionadas	- Uso de Django REST Framework para la exposición de servicios. - Integración del backend con Azure SQL.

Comentarios

Esta decisión mejora la flexibilidad del sistema, permitiendo futuras expansiones a otros clientes (por ejemplo, aplicaciones móviles).